TECHNICAL WHITE PAPER / 12 OCTOBER 2026

ILRN-4-96 vs. causal Transformer

with a later CPU speed-up study

Four layers, 96 features; 96/256-token news tests and a separate fused CPU study.

Independent researcher | rickeyernest1990@gmail.com

ILRN-4-96 stacks four ILRM memory blocks. Part I compares the full network with a Transformer; Part II evaluates its fused CPU execution path.

ORIGINAL CNN/DAILYMAIL RESULTS FIRST

96-token benchmark

ILRN 27.17% vs Transformer 25.83%

Held-out test top-1 accuracy

CPU: 53.3 min vs 15.4 min

ILRN was 3.45x slower in the original CPU run

256-token benchmark

ILRN 27.42% vs Transformer 24.95%

Held-out test top-1 accuracy

CPU: 114.0 min vs 17.7 min

ILRN was about 6.44x slower in the original CPU run

Separate later speed-up study

The ILRN fused CPU path, preserving ILRM mathematics, was tested on a separate 10M-target code-text benchmark. In that separate study it trained 5.19x faster with no observed accuracy drop.

READING GUIDE

How to read this paper

The report is organised in two parts: Part I covers the original CNN/DailyMail architecture comparisons, and Part II covers the later, separate CPU speed-up experiment.

Part I / Original CNN/DailyMail experiments

EXPERIMENT 01

96 tokens

ILRN vs Transformer

EXPERIMENT 02

256 tokens

ILRN vs Transformer

EXPERIMENT 03

Aligned check

96 vs 256 on matched targets

What Part I establishes

01 / CNN NEWS

96-token model comparison

Does ILRN learn better than a parameter-matched causal Transformer at 20M targets?

02 / CNN NEWS

256-token model comparison

Does the same general outcome hold at a longer input window?

03 / CNN NEWS

Target-aligned context analysis

When both context settings are scored on the same targets, what actually changes?

Part II / Separate implementation benchmark

EXPERIMENT 04

Fused CPU kernel

Same ILRM math; different execution

Key result

5.19x faster

10M-target training-only speed-up

Separate code-text benchmark

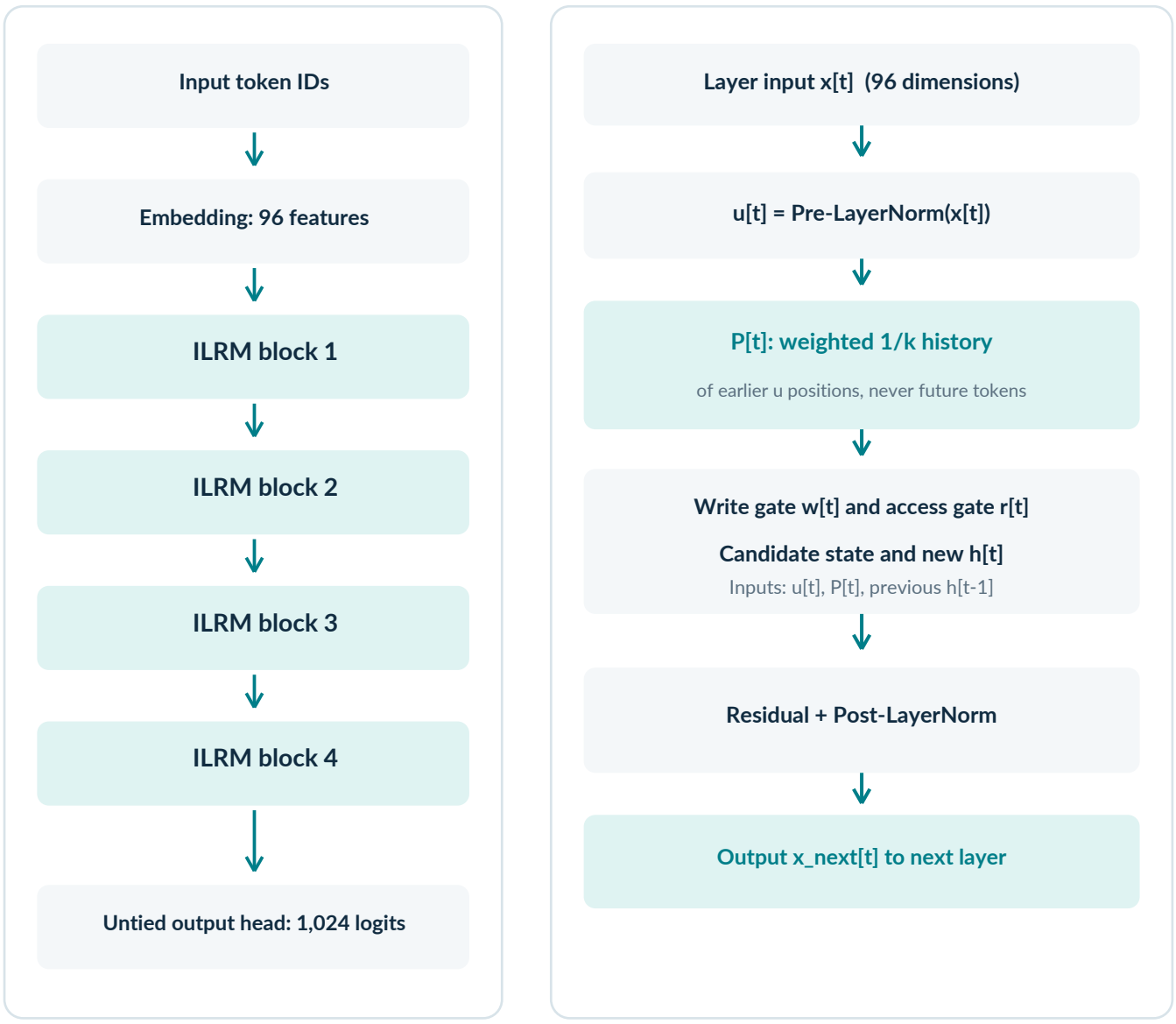
Important boundary

Do not read the speed-up study as a third CNN/DailyMail result. It answers a different question: whether the same ILRN computation can run faster without an observed loss of fidelity.

SYSTEM DESIGN

How the four-layer stack is organised

The left diagram shows information moving through layers in sequence. The right diagram expands the computations of ANY one layer; it is not an extra fifth layer.



Across layers: each ILRN layer contains an ILRM cell. Layer outputs feed the next layer, with separate inverse-lag summaries and recurrent states. Each layer starts a training window with its own state set to zero.

Important: two different directions

Vertical: information moves between layers. Along time: each layer uses earlier token representations and h[t-1]. The mathematical pages next specify every branch precisely.

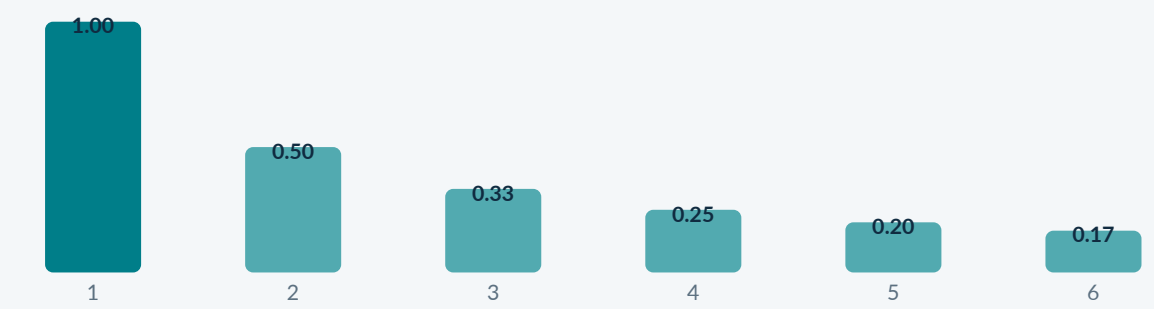
HOW THE MEMORY WORKS

Two types of memory, working together

The ILRM block has an explicit history summary and a recurrent state. They have different jobs, and keeping them separate makes the design easier to understand.

1 The inverse-lag summary: recent past matters more

At position t, previous layer inputs are averaged with weights proportional to 1 / lag. The current position is excluded. The weights are normalised, so the summary is a weighted average rather than a growing sum.



FFT acceleration: the weighted history is a causal convolution with a fixed 1/k kernel. During training, a fast Fourier transform computes that history for a whole window; it does not make the state update parallel.

2 The state h[t]: a learned decision about what to keep

Write gate

w[t] = sigmoid(projected inputs + recurrent contribution)

Candidate c[t] is shaped by an access gate r[t] and the previous state.

h[t] = h[t-1] + w[t] * (c[t] - h[t-1])

In plain English: the model first asks what earlier tokens say (inverse-lag history), then uses its gates to decide how much of its previous internal state to replace. This operation runs sequentially because the next gates depend on the previous state.

MATHEMATICS 01 / NOTATION

The ILRM block, defined precisely

Each of the four layers uses the original ILRM recurrence with separate weights. Here u[t] = LN(x[t]) plays the role of the original paper's input x[t]; t indexes positions and ell indexes layers.

Symbol	Meaning
x_t^(ell)	Input feature vector to layer ell, width d = 96
u_t^(ell)	Pre-normalised current layer input
P_t^(ell)	Weighted history of earlier inputs in this layer
h_t^(ell)	Persistent recurrent state for this layer and window
w_t, r_t	Write and access gate vectors (each width 96)
c_t = tilde h_t	Candidate state for recurrent update

1. PRE-NORMALISATION

$$u_t^{(\ell)} = \mathrm{LN}_{pre}^{(\ell)}(x_t^{(\ell)})$$

This representation is used for the history and gates.

2. HARMONIC HISTORY

$$H_t = \sum_{k=1}^{t} \frac{1}{k}, \qquad P_t^{(\ell)} = \frac{1}{H_t} \sum_{k=1}^{t} \frac{u_{t-k}^{(\ell)}}{k}$$

Set P_0 = 0; the summation never uses the current or a future token.

3. GATE INPUT

$$z_t = [\, u_t \,;\, h_{t-1} \,;\, P_t \,]$$

The concatenated vector has 3d = 288 components.

Boundary condition: every layer begins an independent training window with h_-1 = 0. The history also starts empty. Neither a learned per-lag decay nor a separate LSTM cell is used.

MATHEMATICS 02 / RECURRENT GATING

How the two gates change the state

The write gate chooses how strongly to replace the previous state. The access gate controls which components of the previous state the candidate calculation may use.

4. WRITE AND ACCESS GATES

$$w_t = \sigma(W_w z_t + b_w), \quad r_t = \sigma(W_r z_t + b_r)$$

Sigmoid is elementwise; each gate is a vector in [0, 1]^96.

5. ACCESS-GATED CANDIDATE

$$\tilde{h}_t = \tanh(W_c[\, u_t \,;\, r_t \odot h_{t-1} \,;\, P_t \,] + b_c)$$

The access gate acts on the previous recurrent state, not on the history sum.

6. STATE UPDATE

$$h_t = (1 - w_t) \odot h_{t-1} + w_t \odot \tilde{h}_t$$

A gated interpolation; h_t still depends on h_(t-1), so time steps remain sequential.

7. RESIDUAL AND POST-NORMALISATION

$$x_t^{(\ell+1)} = \mathrm{LN}_{post}^{(\ell)}\left(x_t^{(\ell)} + h_t^{(\ell)}\right)$$

This is the layer-to-layer connection used four times.

How to read the state update

If a write-gate coordinate approaches 0, that coordinate of h[t-1] is largely retained. If it approaches 1, the new candidate replaces it. Because the gates depend on the previous state, a generic associative parallel scan cannot reproduce this exact nonlinear recurrence without changing the model.

MATHEMATICS 03 / COMPLETE STACK

From token IDs to next-token logits

The ILRM block equations above are repeated in a four-layer stack. All four layers see a whole token window during training, but each layer has its own recurrent state and its own parameters.

8. INPUT EMBEDDING

$$x_t^{(1)} = \text{Embedding}(s_t) \in \mathbb{R}^{96}$$

Token s_t is an ID in the 1,024-piece BPE vocabulary.

9. FOUR STACKED BLOCKS

$$x^{(5)} = B_4(B_3(B_2(B_1(x^{(1)}))))$$

Each B_l denotes pre-normalisation, memory, gates, recurrence, residual and post-LN.

10. UNTIED PREDICTION HEAD

$$\text{logits}_t = W_{out} x_t^{(5)} + b_{out}$$

The head has different trainable weights from the embedding table.

Exact trainable-parameter accounting

Component	Parameters
Input embedding: 1,024 x 96	98,304
Single block: three 96 x 288 projections + biases + two LNs	83,616
Four independent blocks	334,464
Untied output linear: 96 x 1,024 + bias	99,328
TOTAL: ILRN-4-96	532,096

The same parameter count applies at context 96 and context 256; changing window length changes computation, not learned matrix sizes.

MATHEMATICS 04 / FAST HISTORY

Why FFT accelerates inverse-lag memory

The inverse-lag history depends only on current-layer inputs, not previous hidden states. This allows all P[t] values in a training window to be computed by one causal convolution.

11. FIXED CAUSAL CONVOLUTION KERNEL

$$q_0 = 0, \qquad q_k = \frac{1}{k} \quad (1 \le k < T)$$

The zero at lag 0 prevents the current token from entering its own history.

12. PARALLEL HISTORY NUMERATOR

$$N_t = \sum_{k=0}^{t} q_k u_{t-k} = (q * u)_t$$

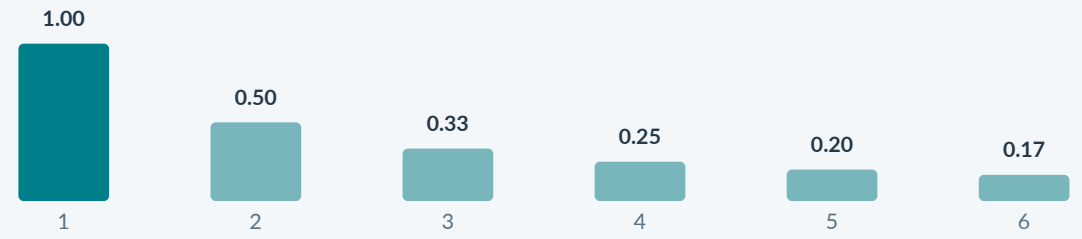
One convolution computes the numerator at all sequence positions.

13. FFT AND HARMONIC NORMALISATION

$$N = \mathrm{IFFT}(\mathrm{FFT}(q) \cdot \mathrm{FFT}(u)), \qquad P_t = N_t / H_t$$

Pad for linear convolution with n >= 2T - 1; discard padded positions.

A visual example: unnormalised weights at lags 1-6



Crucial distinction: FFT parallelises history construction. It does not parallelise h[t], because the gates and candidate depend on h[t-1]. The later fused CPU kernel keeps this dependency and reduces its execution overhead instead.

MATHEMATICS 05 / CAUSAL TRANSFORMER

The comparison model is fully specified

The news-study baseline was a four-layer causal decoder-style Transformer, with width 96, four attention heads (24 features each), GELU feed-forward width 237, sinusoidal positions and no dropout.

14. INPUT AND QUERIES / KEYS / VALUES

$$X^{(0)} = E(s) + PE, \quad Z = \mathrm{LN}_1(X), \quad Q = ZW_Q,\ K = ZW_K,\ V = ZW_V$$

Sinusoidal position features are added at input; Q, K and V are linear projections.

15. CAUSAL SCALED DOT-PRODUCT ATTENTION

$$\mathrm{head}_j = \mathrm{softmax}\left(\frac{Q_j K_j^T}{\sqrt{24}} + M\right) V_j$$

Mask M[t,s] = negative infinity if s > t, otherwise 0.

16. ATTENTION AND FEED-FORWARD RESIDUALS

$$U = X + \mathrm{MHA}(\mathrm{LN}_1(X))$$

The attention output is added to the block input.

17. FEED-FORWARD SUBLAYER

$$Y = U + W_2\,\mathrm{GELU}(W_1\,\mathrm{LN}_2(U) + b_1) + b_2$$

The feed-forward path maps 96 -> 237 -> 96 features.

18. FINAL OUTPUT

$$\mathrm{logits}_t = W_{out}\,\mathrm{LN}_{final}(X_t^{(4)}) + b_{out}$$

Total trainable parameters: 531,700, compared with 532,096 in ILRN-4-96.

MATHEMATICS 06 / EVALUATION RULES

What was trained, timed and scored

Metrics are per next-token target, not whole English words. Each data study used its own fixed held-out windows. The 96- and 256-token news comparisons each exposed their two models to 20M targets.

19. MEAN TEACHER-FORCED CROSS-ENTROPY

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid x_{\le i})$$

The true next-token ID y_i is used to score each model prediction.

20. PERPLEXITY AND CLASSIFICATION METRICS

$$\mathrm{PPL} = \exp(\mathcal{L}), \quad \mathrm{Top1} = \frac{1}{N}\sum_i \mathbf{1}[\arg\max p_i = y_i]$$

Top-5 credits a target whenever its ID appears among the five highest probabilities.

Controlled comparisons in the original news study

Criterion	Setting
Tokenizer	Train-only SentencePiece BPE; 1,024 subword IDs
Model width / layers	96 features / 4 layers for both architectures
Training exposure	20M next-token target positions per model
Optimiser	AdamW, LR 0.001, weight decay 0.01, clip 1.0
Regularisation / schedule	Dropout 0; no warm-up or decay
Checkpoint frequency	Every 2M targets; single seed 20261011
Scoring	Fixed validation subsets; separate held-out test subsets

Equal targets and similar parameter counts do NOT imply equal floating-point operations, elapsed time or tuning quality. The conclusions are restricted to these settings.

MATHEMATICS 07 / REMAINING DEFINITIONS

Normalisation and model dimensions

To make the equations implementation-ready, this page spells out the standard supporting operations. The exact numerical epsilon for the original news implementation is not recorded in the supplied report.

29. LAYERNORM OVER THE 96 FEATURES

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu(x)}{\sqrt{\mathrm{Var}(x) + \varepsilon}} + \beta$$

Gain gamma and bias beta are learned; the reference code uses the PyTorch LayerNorm default epsilon.

30. CONVERT LOGITS TO TOKEN PROBABILITIES

$$p_\theta(y = j \mid x) = \frac{\exp(a_j)}{\sum_{v=1}^{1024} \exp(a_v)}$$

The output head gives 1,024 logits a_j; softmax gives a categorical distribution.

31. TOP-FIVE TOKEN ACCURACY

$$\mathrm{Top5} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{y_i \in \mathrm{five_largest}(p_i)\}$$

The ranking is performed over subword IDs, not complete words.

Tensor dimensions in the reference implementation

Tensor / weight	Dimensions
Input tokens	batch x time (integer token IDs)
u, P, h, w, r and candidate	batch x time x 96
z gate-input concatenation	batch x time x 288
W_w, W_r, W_c	96 x 288 each
Logits	batch x time x 1,024

A standard fixed sinusoidal positional encoding is used by the Transformer comparator, not ILRN. The news source report does not give its exact elementwise implementation, so no undocumented variant is asserted here.

EXPERIMENT 01 / CNN/DAILYMAIL

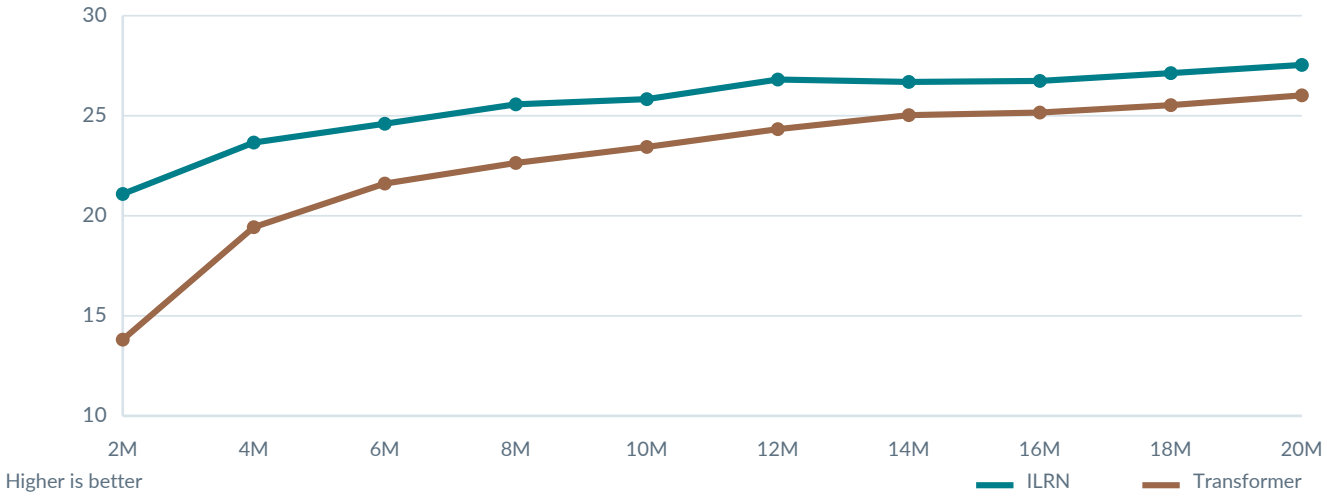
The 96-token benchmark

Question: under matched training exposure and almost identical parameter counts, how does ILRN compare with a causal Transformer?

20M targets / model | 96-token context | ~532K parameters

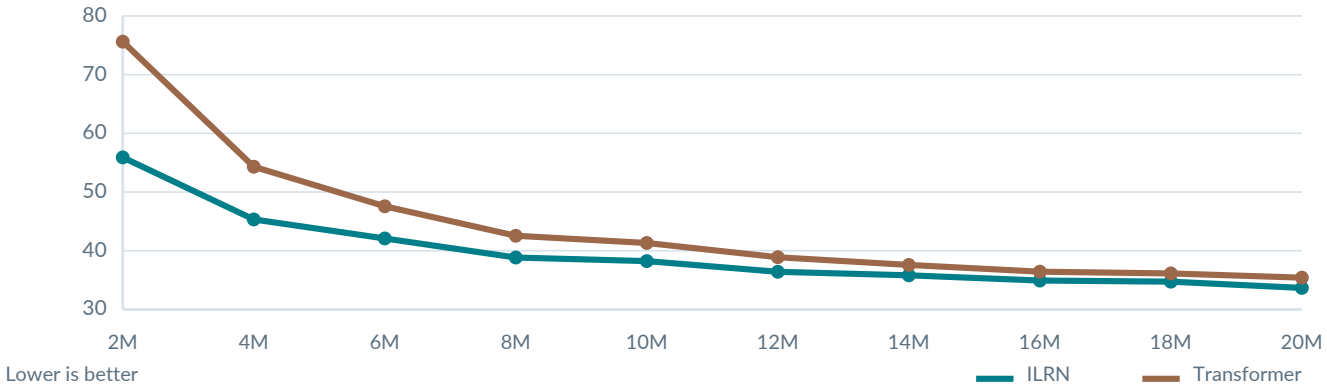
Both networks used the same 1,024-piece BPE tokenizer, the same news training stream, and the same 20M next-token prediction positions. The models were trained from scratch with one seed and evaluated on shared held-out windows.

Validation top-1 accuracy (%)



Higher is better

Validation perplexity



Lower is better

Final held-out test at 20M targets

ILRN: 27.17% top-1 | 48.47% top-5 | PPL 34.23

Transformer: 25.83% top-1 | 47.66% top-5 | PPL 36.05

CPU training time: ILRN 53.3 min; Transformer 15.4 min (3.45x gap).

EXPERIMENT 02 / CNN/DAILYMAIL

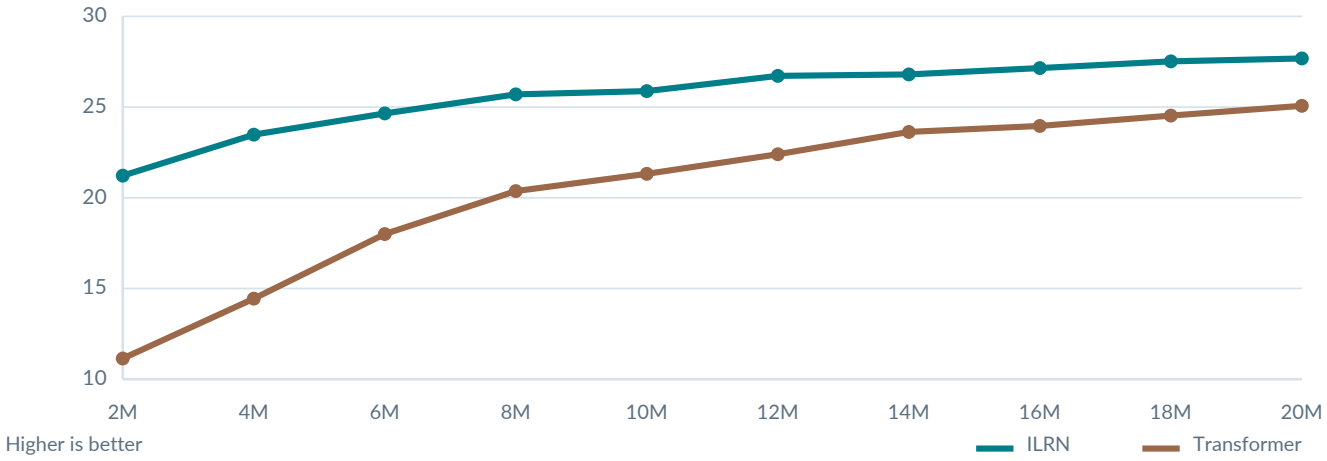
Does the result hold at 256 tokens?

The architecture, parameter budget and 20M-token exposure remained the same. The context length increased from 96 to 256; each batch used six independent windows.

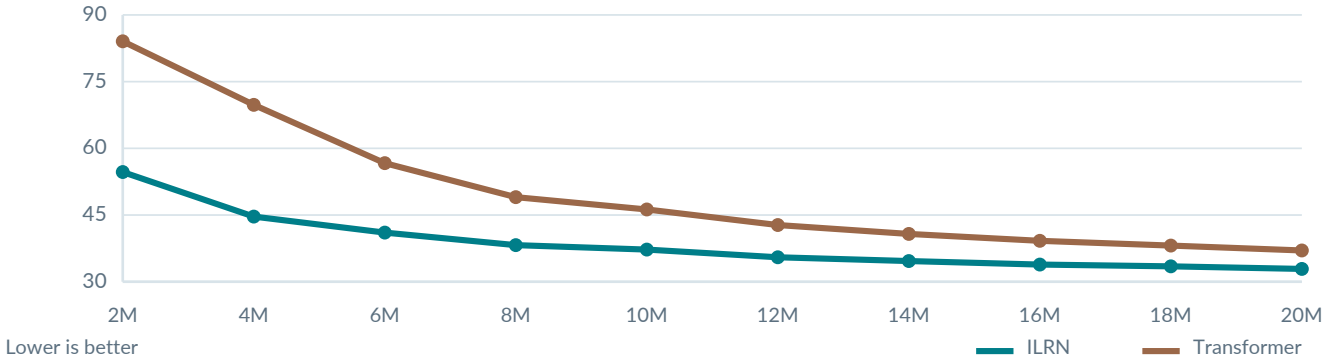
20M targets / model | 256-token context | 6 windows x 256

A longer window lets each model see more preceding tokens, but it also changes compute and training-window resets. The aim here was to repeat the architecture comparison, not to claim that context length alone caused every difference.

Validation top-1 accuracy (%)



Validation perplexity



Final held-out test at 20M

ILRN: 27.42% top-1 | 48.84% top-5 | PPL 33.79

Transformer: 24.95% top-1 | 46.89% top-5 | PPL 37.84

CPU training time: ILRN 114.0 min; Transformer 17.7 min (about 6.44x).

EXPERIMENT 03 / CNN/DAILYMAIL

A fairer 96 vs 256 comparison

The earlier validation curves use different windows. This follow-up scores the same 15,264 validation targets for each context, limiting one important source of comparison bias.

At 20M training targets, on aligned validation positions

Metric	Context 96	Context 256	256 minus 96
ILRN · top-1	27.56%	27.50%	-0.06 pp
ILRN · perplexity	33.59	32.51	-1.08 (better)
Transformer · top-1	26.05%	24.95%	-1.10 pp
Transformer · perplexity	35.33	36.40	+1.07 (worse)

How to interpret the aligned result

ILRN

More context led to lower perplexity on these matched targets, while top-1 accuracy was essentially unchanged.

Transformer

The longer-context model scored worse on both aligned top-1 accuracy and perplexity in this configuration.

Not a pure length ablation

The 96- and 256-context models were trained separately. Window boundaries, reset positions and grouping also changed, so the observed differences cannot be credited solely to extra preceding tokens.

PART II / SEPARATE SPEED BENCHMARK

A different question: can ILRN run faster?

This last experiment did not compare ILRN with a Transformer. It compared two ways of executing the same four-layer ILRN network with unchanged ILRM equations.

Dataset distinction

The 10M-target benchmark used source code from installed Python packages, split into disjoint package groups for training, validation and test. It was not CNN/DailyMail and not WikiText-103. Quality scores must not be compared directly with the news-language experiments.

Matched conditions for the two implementations

CONTROL	BOTH VERSIONS
Network	ILRN: 4 ILRM blocks; width 96; ~532K parameters
Input	256-token context; six windows per batch
Exposure	10M distinct training target positions per model
Tokeniser	1,024-piece training-only SentencePiece BPE
Optimiser	AdamW; same initial weights, tokens and training settings
Evaluation	40,960 fixed validation and 40,960 independent test targets
Milestones	2M, 4M, 6M, 8M and 10M
CPU	Four threads on the measured benchmark machine

The scientific goal

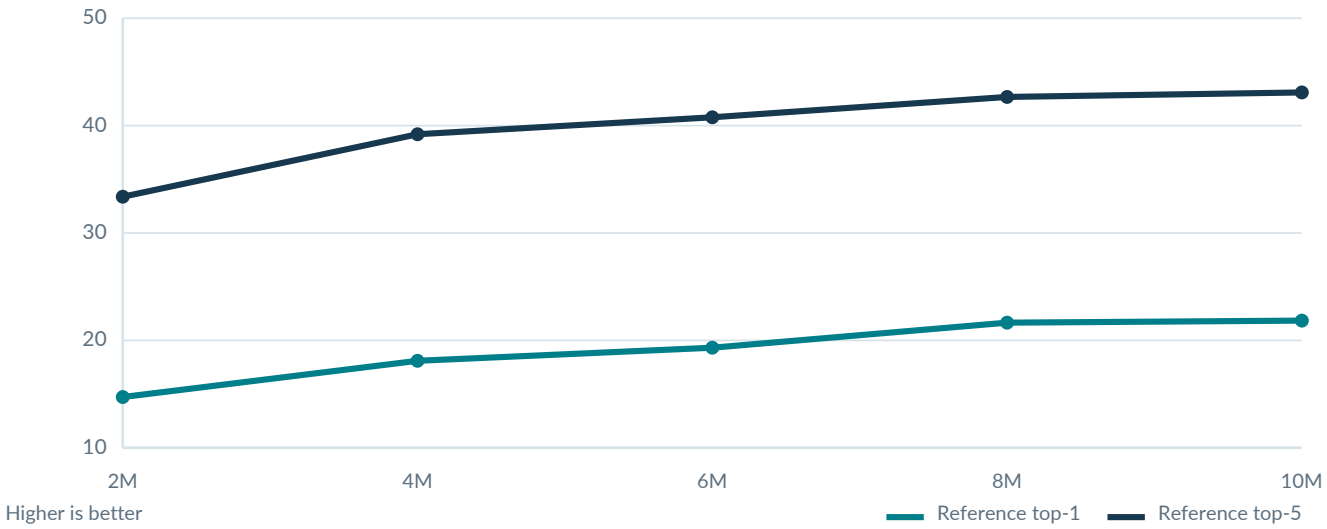
If the fused version finishes sooner while achieving essentially identical losses, predictions and parameter updates, that supports an implementation speed-up - not a new model architecture.

PART II / ACCURACY CHECK

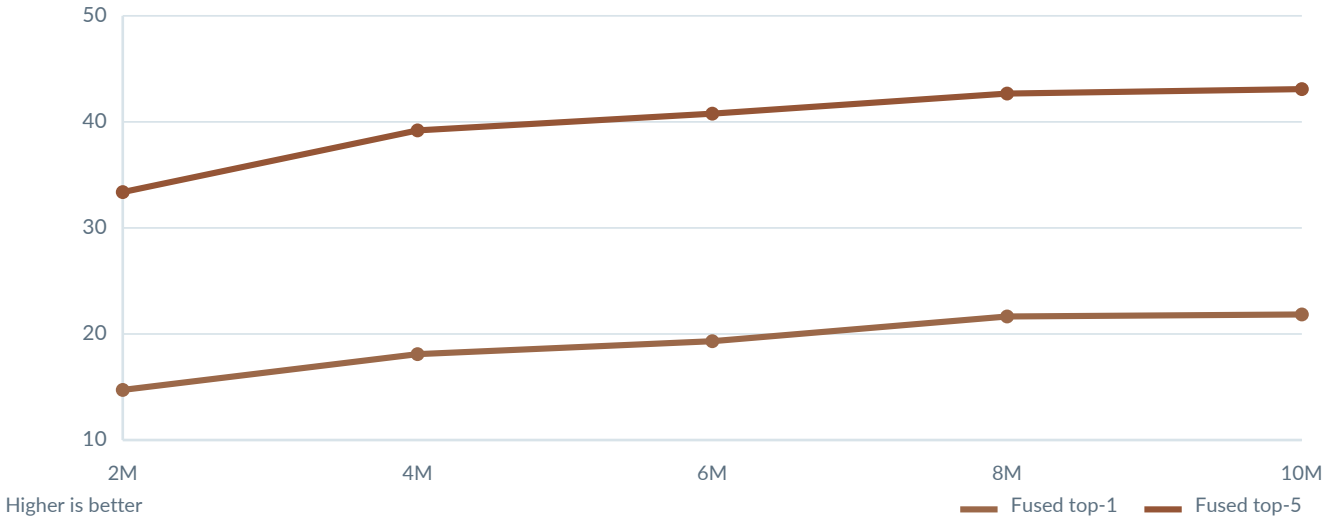
Did the faster code damage accuracy?

Two separate panels use the same scale and fixed held-out targets. The plotted checkpoint points lie on top of each other at the precision reported.

Graph A / normal ILRN - validation accuracy (%)



Graph B / fused ILRN - validation accuracy (%)



Measured top-1 difference at every checkpoint

0.0000 percentage points (to reported precision)

The results support no observed accuracy drop in this run. They do not prove bitwise identity or rule out differences on other hardware, seeds or datasets.

PART II / TRAINING SPEED

The same work, in much less CPU time

Both models completed 10M next-token training targets with forward propagation, backpropagation and AdamW updates. Times below exclude first-time compilation, evaluation, data preparation and checkpoint I/O.

NORMAL TRAINING ONLY

56.8 min

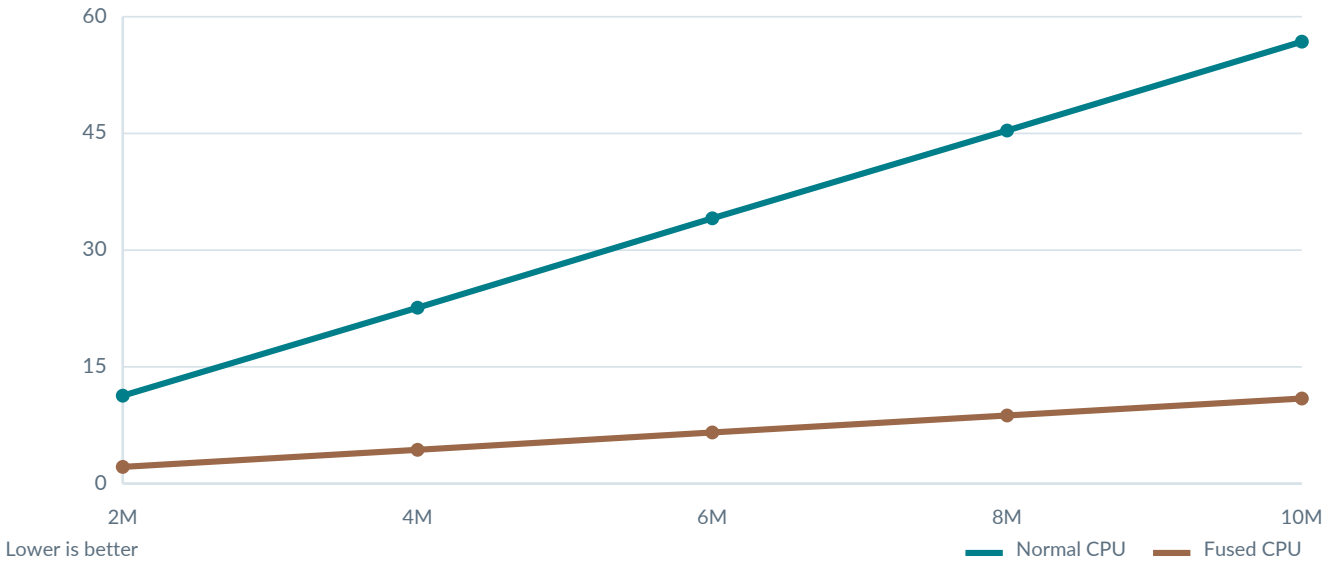
3,407.70 sec over 10M targets

FUSED TRAINING ONLY

10.9 min

655.97 sec over 10M targets

Cumulative training-only wall time (minutes)



TARGETS	NORMAL	FUSED	SPEED-UP
2M	677.8 s	128.6 s	5.27x
4M	678.4 s	131.7 s	5.15x
6M	689.2 s	133.9 s	5.15x
8M	676.9 s	130.9 s	5.17x
10M	685.4 s	130.9 s	5.24x

Overall 5.19x faster; CPU-dependent and excludes compilation, evaluation and I/O.

PART II / IMPLEMENTATION DETAILS

What the fused kernel actually changes

The weights, inverse-lag memory, recurrent equations and four-layer stack remain the same. Only the CPU execution plan changes.

NORMAL EXECUTION

1. Separate input / history projections
2. Repeated gate work per token
3. Python / framework autograd steps
4. Backward through time across many ops

FUSED CPU EXECUTION

1. Pack 3 gate projections together
2. Precompute input-only terms
3. Compiled recurrent gate loop
4. Custom backward through time

The update is still the same recurrence

```
w[t] = sigmoid(pre_w[t] + Ww_h * h[t-1])
r[t] = sigmoid(pre_r[t] + Wr_h * h[t-1])
h[t] = h[t-1] + w[t] * (c[t] - h[t-1])
```

The candidate c[t] is also computed with the original access-gated previous state. The inputs to all three gates are combined into a single preprojection. A CPU JIT-compiled kernel handles the remaining sequential steps and their gradients.

Why this can help without parallelising time

The speed comes mainly from reducing many small framework operations, not from replacing the state update with a different mathematical recurrence.

FUSED KERNEL / MATHEMATICAL EQUIVALENCE

What is precomputed - and what is not

The fused implementation changes execution, not trained weights or recurrent equations. Gate matrices are split by the three concatenated input groups: current input u, prior state h, and history P.

21. PARTITION EACH GATE MATRIX

$$W_g = [\, W_g^u \,;\, W_g^h \,;\, W_g^P \,], \qquad g \in \{w, r, c\}$$

Every matrix still multiplies the same original features in the same order.

22. PACK THE NONRECURRENT TERMS ONCE

$$A_{g,t} = W_g^u u_t + W_g^P P_t + b_g$$

The three A arrays are computed together by one packed projection.

23. EXECUTE THE SAME SEQUENTIAL RECURRENCE

$$w_t = \sigma(A_{w,t} + W_w^h h_{t-1}), \quad r_t = \sigma(A_{r,t} + W_r^h h_{t-1})$$

The gates still consume the previous state; the time loop is not mathematically parallel.

24. CANDIDATE AND UPDATE

$$c_t = \tanh(A_{c,t} + W_c^h (r_t \odot h_{t-1})), \quad h_t = h_{t-1} + w_t \odot (c_t - h_{t-1})$$

These are exactly the earlier ILRM equations rewritten by separating linear terms.

Supported portability

The accompanying prototype targets CPU float32 and Numba-supported processors. It is not an optimised universal native kernel; speed is hardware-, thread- and workload-dependent.

FUSED KERNEL / BACKWARD PASS

How gradients still reach every gate

The custom kernel uses backpropagation through time. It computes gradients of the sequential recurrent state and returns them to PyTorch, which differentiates the packed input projection, layer norms, FFT history and embedding.

25. UPSTREAM STATE GRADIENT AND CANDIDATE/WRITE BRANCHES

$$g_t = \frac{\partial \mathcal{L}}{\partial h_t}, \quad d_{c,t} = g_t \odot w_t \odot (1 - c_t^2)$$

The candidate preactivation uses the derivative of tanh.

26. WRITE-GATE DERIVATIVE

$$d_{w,t} = g_t \odot (c_t - h_{t-1}) \odot w_t \odot (1 - w_t)$$

The derivative includes how much replacing the previous state changes h_t.

27. ACCESS-GATE DERIVATIVE

$$m_t = (W_c^h)^T d_{c,t}, \quad d_{r,t} = (m_t \odot h_{t-1}) \odot r_t \odot (1 - r_t)$$

The access gate affects the candidate through r_t multiplied by the previous state.

28. GRADIENT FLOWING TO THE PRECEDING TIME STEP

$$g_{t-1} = g_t \odot (1 - w_t) + m_t \odot r_t + (W_w^h)^T d_{w,t} + (W_r^h)^T d_{r,t}$$

Add gradients from the layer output at t-1 when iterating backward through time.

The recurrent weight gradients accumulate outer products such as d_w,t times h_(t-1) transposed; the candidate recurrent projection accumulates d_c,t times (r_t * h_(t-1)) transposed. PyTorch handles gradient flow into the packed A projection and inverse-lag history.

Numerical fidelity is empirical

The code-text study reported matching accuracy and small parameter differences. These results do not establish bitwise identity or fidelity on the original news benchmark.

PART II / FIDELITY CHECKS

Beyond accuracy: do the models still agree?

Matching accuracy alone could hide differences. The final experiment also compared uncertainty, parameter drift and selected training diagnostics.

FINAL MEASUREMENT	NORMAL	FUSED
Validation perplexity	50.571361	50.571362
Validation top-1	21.8408%	21.8408%
Validation top-5	43.0908%	43.0908%
Independent test perplexity	40.665170	40.665176
Independent test top-1	24.0698%	24.0698%
Independent test top-5	47.1802%	47.1802%
Final mean pre-clip grad norm	0.918211	0.918211
Write-gate saturation	22.3328%	22.3328%

LARGEST ABSOLUTE WEIGHT GAP

1.15e-5

At the final 10M checkpoint

MEAN ABSOLUTE WEIGHT GAP

6.28e-7

Corresponding parameters after 10M

What this establishes

The two implementations followed nearly identical learning trajectories on a fixed code-text dataset, with no observed sampled accuracy loss. Small floating-point differences remain; equivalence on CNN/DailyMail is untested.

INTERPRETATION

What we can - and cannot - conclude

The strongest scientific reading keeps model capability, context effects and CPU execution performance as separate claims.

SUPPORTED	ILRN beat the tested matched Transformer on held-out CNN/DailyMail next-token metrics at contexts 96 and 256.
SUPPORTED	A 256-context ILRN gave lower aligned validation perplexity than its 96-context counterpart in the recorded study.
SUPPORTED	The fused recurrent CPU path cut measured 10M-target training time by 5.19x on the separate code-text benchmark.
SUPPORTED	In that matched run, fused and normal ILRN had identical recorded top-1/top-5 accuracy at five validation checkpoints and on the final independent test.
NOT SHOWN	The fused implementation preserves CNN/DailyMail accuracy on the original trained checkpoint or on every processor.
NOT SHOWN	The model is generally better than Transformers, or capable of strong conversation / reasoning at larger scales.

Most useful next replication

Run the fused code on the original CNN/DailyMail training implementation and checkpoints, and report speed plus token-level fidelity on the same held-out news targets. Do not extrapolate the 5.19x figure from the code-text benchmark.

DATA APPENDIX A

Complete 96-context checkpoints

All numbers below come from the original CNN/DailyMail 20M study. The rows are matched validation evaluations, not test scores.

TARGETS	ILRN PPL	TRANS. PPL	ILRN TOP-1	TRANS. TOP-1
2M	55.90	75.62	21.09%	13.81%
4M	45.33	54.31	23.66%	19.43%
6M	42.09	47.57	24.60%	21.61%
8M	38.84	42.55	25.57%	22.64%
10M	38.23	41.32	25.83%	23.44%
12M	36.41	38.89	26.81%	24.33%
14M	35.81	37.58	26.69%	25.03%
16M	34.92	36.43	26.74%	25.16%
18M	34.73	36.13	27.13%	25.53%
20M	33.66	35.41	27.54%	26.02%

Held-out test after 20M

METRIC	ILRN	TRANSFORMER
Top-1 accuracy	27.17%	25.83%
Perplexity	34.23	36.05

Source: supplied CNN/DailyMail report, sections 08-10. CPU training: ILRN 3,195.8 s vs Transformer 925.2 s. One seed and fixed test-window subset.

DATA APPENDIX B

Complete 256-context checkpoints

All numbers below come from the 256-context CNN/DailyMail addendum. Models received the same 20M target positions as one another.

TARGETS	ILRN PPL	TRANS. PPL	ILRN TOP-1	TRANS. TOP-1
2M	54.67	84.07	21.22%	11.14%
4M	44.65	69.78	23.48%	14.44%
6M	41.05	56.67	24.65%	18.00%
8M	38.23	49.02	25.70%	20.37%
10M	37.23	46.25	25.88%	21.32%
12M	35.50	42.75	26.72%	22.40%
14M	34.65	40.75	26.80%	23.63%
16M	33.86	39.20	27.15%	23.96%
18M	33.46	38.13	27.52%	24.53%
20M	32.88	37.03	27.68%	25.07%

Held-out test after 20M

METRIC	ILRN	TRANSFORMER
Top-1 accuracy	27.42%	24.95%
Perplexity	33.79	37.84

Source: supplied CNN/DailyMail report, sections 15-19. CPU training: ILRN 114.0 min vs Transformer 17.7 min. One seed and a fixed 320-window test subset.

DATA APPENDIX C

10M fused-kernel checkpoint audit

These measurements come from the separate offline code-text benchmark and the saved paired checkpoint CSV, not the news-language experiment.

TARGETS	REF. TIME	FUSED TIME	SPEED-UP	TOP-1 (BOTH)
2M	677.8s	128.6s	5.27x	14.7241%
4M	678.4s	131.7s	5.15x	18.0981%
6M	689.2s	133.9s	5.15x	19.3188%
8M	676.9s	130.9s	5.17x	21.6504%
10M	685.4s	130.9s	5.24x	21.8408%

Source and reproducibility boundary

News experiments: supplied 20-page original report, including the context-256 addendum; table values are from recorded checkpoint and test summaries in that report. Study date: 11 October 2026.

Code-text acceleration: supplied 24-page appended report; 10M-token matched benchmark summary; **comparison.csv**, **reference_metrics.csv**, and **fused_metrics.csv**. Study date: 12 October 2026.

Archived implementation filenames: **ilrm_fused_dropin.py** (opt-in wrapper), **ilrm_fused_projection_gate_benchmark.py** (compiled forward / backward), and **ilrm_wikitext10m_paired_v3.py** (runner; despite the filename, the completed offline run used code-text, not WikiText-103).

Limits to reproduction

The code-text archive did not redistribute the package-derived source corpus. It retained tokeniser and token-stream fingerprints. Exact benchmark reruns require matching software, data and CPU conditions. This white paper re-presents the supplied experimental findings; it is not an independent replication.

REPRODUCIBILITY / EXECUTABLE CODE

Three files to recreate and test the model

The code in the following appendix is provided as a tested, standalone reconstruction of the reported mathematics. It is NOT the original CNN/DailyMail training program. It requires torch; the fused option additionally requires numba and numpy.

Recommended reproduction workflow

Step	Action
1	Install the versions listed in requirements.txt
2	Save ilrn_reference.py, ilrn_fused_cpu.py and verify_ilrn.py
3	Run python ilrn_reference.py to see a 4-layer training step
4	Run python verify_ilrn.py for forward, gradient and update checks
5	Pass matching weights and tokens to both implementations
6	Record CPU hardware, thread counts and compilation time

Scope of the included implementation

It implements a single ILRM block, four-layer ILRN model, FFT inverse-lag history, the optional fused CPU forward/backward, and numerical parity tests. It does not download the original licensed news corpus, reproduce original checkpoints or implement a separate streaming-inference API.

Companion source files

ilrn_reference.py: full reference block and four-layer stack. ilrn_fused_cpu.py: packed gates, Numba forward/backward and opt-in switch. verify_ilrn.py: correctness checks. requirements.txt: dependencies.

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

ILRM block and four-layer ILRN stack

Torch reference, causal FFT memory and one AdamW step

ilrn_reference.py Lines 1-40 of 97

```
"""Four-layer ILRN-4-96 mathematical reference.
Not the original news training code.

Requires: torch. Input token tensor: [batch, context].
Outputs logits [batch, context, vocab].
State h is reset per window. No separate streaming inference API is included.
"""
import torch
from torch import nn
from torch.nn import functional as F


class ILRMBlock(nn.Module):
    """Inverse-lag history and the original access/write recurrence."""

    def __init__(self, width=96):
        super().__init__()
        self.width = width
        self.pre = nn.LayerNorm(width)
        self.post = nn.LayerNorm(width)
        # Input order of every gate weight: [u_t ; h_(t-1) ; P_t].
        self.Ww = nn.Parameter(torch.randn(width, 3 * width) * 0.035)
        self.Wr = nn.Parameter(torch.randn(width, 3 * width) * 0.035)
        self.Wc = nn.Parameter(torch.randn(width, 3 * width) * 0.035)
        self.bw = nn.Parameter(torch.zeros(width))
        self.br = nn.Parameter(torch.zeros(width))
        self.bc = nn.Parameter(torch.zeros(width))

    @staticmethod
    def history(u):
        """Causal harmonic 1/k summary. Current token has zero weight."""
        batch, steps, width = u.shape
        q = torch.zeros(steps, device=u.device, dtype=u.dtype)
        if steps > 1:
            lags = torch.arange(1, steps, device=u.device, dtype=u.dtype)
            q[1:] = lags.reciprocal()
        harmonics = torch.cumsum(q, dim=0).clamp_min(1.0)
        n = 1 << ((2 * steps - 2).bit_length())
        transformed = torch.fft.rfft(u.transpose(1, 2), n=n, dim=-1)
        kernel = torch.fft.rfft(q, n=n)
```

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

ILRM block and four-layer ILRN stack

Torch reference, causal FFT memory and one AdamW step

ilrn_reference.py Lines 41-80 of 97

```
        result = torch.fft.irfft(transformed * kernel, n=n, dim=-1)
        return result[:, :, :steps].transpose(1, 2) / harmonics[None, :, None]

    def forward(self, x):
        u = self.pre(x)
        p = self.history(u)
        batch, steps, width = u.shape
        h = u.new_zeros((batch, width))
        sequence = []
        for t in range(steps):
            z = torch.cat((u[:, t], h, p[:, t]), dim=-1)
            w = torch.sigmoid(F.linear(z, self.Ww, self.bw))
            r = torch.sigmoid(F.linear(z, self.Wr, self.br))
            v = torch.cat((u[:, t], r * h, p[:, t]), dim=-1)
            candidate = torch.tanh(F.linear(v, self.Wc, self.bc))
            h = (1 - w) * h + w * candidate
            sequence.append(h)
        return self.post(x + torch.stack(sequence, dim=1))


class StackedILRN(nn.Module):

    """Four independent ILRM blocks, one embedding, and an untied output head."""

    def __init__(self, vocab=1024, width=96, layers=4):
        super().__init__()
        self.embedding = nn.Embedding(vocab, width)
        self.blocks = nn.ModuleList([ILRMBlock(width) for _ in range(layers)])
        self.head = nn.Linear(width, vocab)

    def forward(self, tokens):
        x = self.embedding(tokens)
        for layer in self.blocks:
            x = layer(x)
        return self.head(x)


def demo_step(context=256, batch=6, seed=20261011):
    torch.manual_seed(seed)
    model = StackedILRN()
```

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

ILRM block and four-layer ILRN stack

Torch reference, causal FFT memory and one AdamW step

ilrn_reference.py Lines 81-97 of 97

```
    tokens = torch.randint(0, 1024, (batch, context))
    labels = torch.randint(0, 1024, (batch, context))
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
    optimizer.zero_grad(set_to_none=True)
    logits = model(tokens)
    loss = F.cross_entropy(logits.reshape(-1, 1024), labels.reshape(-1))
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    return model, float(loss.detach())


if __name__ == '__main__':
    torch.set_num_threads(2)
    model, loss = demo_step(context=32, batch=2)
    count = sum(p.numel() for p in model.parameters())
    print(f'parameters={count:,}; loss={loss:.4f}')
```

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

Fused CPU execution for ILRM blocks

Packed projections, Numba forward/backward and opt-in wrapper

ilrn_fused_cpu.py Lines 1-40 of 135

```
"""CPU float32 opt-in fused ILRM recurrence, preserving reference parameters.

Requires torch, numpy, numba. Fuses the input-only projections and uses
Numba JIT for the recurrent forward/backward. The history remains FFT based.
The reference model's state_dict structure is unchanged by enable_fused_cpu.
"""
from types import MethodType
import numpy as np
import numba as nb
import torch
from torch.nn import functional as F


@nb.njit(parallel=True, cache=True)
def _forward(pre, ww, wr, wc):
    batch, steps, triple = pre.shape
    width = triple // 3
    hs = np.empty((batch, steps, width), np.float32)
    gates_w = np.empty_like(hs)
    gates_r = np.empty_like(hs)
    candidates = np.empty_like(hs)
    for b in nb.prange(batch):
        h = np.zeros(width, np.float32)
        for t in range(steps):
            for j in range(width):
                sw = pre[b, t, j]
                sr = pre[b, t, width + j]
                for k in range(width):
                    sw += ww[j, k] * h[k]
                    sr += wr[j, k] * h[k]
                gates_w[b, t, j] = 1.0 / (1.0 + np.exp(-sw))
                gates_r[b, t, j] = 1.0 / (1.0 + np.exp(-sr))
            for j in range(width):
                z = pre[b, t, 2 * width + j]
                for k in range(width):
                    z += wc[j, k] * (gates_r[b, t, k] * h[k])
                candidate = np.tanh(z)
                candidates[b, t, j] = candidate
                hs[b, t, j] = h[j] + gates_w[b, t, j] * (candidate - h[j])
            for j in range(width):
```

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

Fused CPU execution for ILRM blocks

Packed projections, Numba forward/backward and opt-in wrapper

ilrn_fused_cpu.py Lines 41-80 of 135

```
                h[j] = hs[b, t, j]
    return hs, gates_w, gates_r, candidates


@nb.njit(parallel=True, cache=True)
def _backward(grad, hs, gates_w, gates_r, candidates, ww, wr, wc):
    batch, steps, width = hs.shape
    gp = np.empty((batch, steps, 3 * width), np.float32)
    gww = np.zeros((batch, width, width), np.float32)
    gwr = np.zeros((batch, width, width), np.float32)
    gwc = np.zeros((batch, width, width), np.float32)
    for b in nb.prange(batch):
        following = np.zeros(width, np.float32)
        zeros = np.zeros(width, np.float32)
        for t in range(steps - 1, -1, -1):
            prev = zeros if t == 0 else hs[b, t - 1, :]
            gprev = np.empty(width, np.float32)
            d_c = np.empty(width, np.float32)

            for j in range(width):
                upstream = grad[b, t, j] + following[j]
                w = gates_w[b, t, j]
                candidate = candidates[b, t, j]
                d_c[j] = upstream * w * (1.0 - candidate * candidate)
                gp[b, t, j] = upstream * (candidate - prev[j]) * w * (1.0 - w)
                gprev[j] = upstream * (1.0 - w)
            dm = np.zeros(width, np.float32)
            for j in range(width):
                dc = d_c[j]
                gp[b, t, 2 * width + j] = dc
                for k in range(width):
                    dm[k] += wc[j, k] * dc
                    gwc[b, j, k] += dc * (gates_r[b, t, k] * prev[k])
            for k in range(width):
                r = gates_r[b, t, k]
                gprev[k] += dm[k] * r
                gp[b, t, width + k] = dm[k] * prev[k] * r * (1.0 - r)
            for j in range(width):
                dw = gp[b, t, j]
                dr = gp[b, t, width + j]
```

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

Fused CPU execution for ILRM blocks

Packed projections, Numba forward/backward and opt-in wrapper

ilrn_fused_cpu.py Lines 81-120 of 135

```
                for k in range(width):
                    gprev[k] += ww[j, k] * dw + wr[j, k] * dr
                    gww[b, j, k] += dw * prev[k]
                    gwr[b, j, k] += dr * prev[k]
            following = gprev
    return gp, gww, gwr, gwc


class _FusedRecurrence(torch.autograd.Function):
    @staticmethod
    def forward(ctx, pre, ww, wr, wc):
        if pre.device.type != 'cpu' or pre.dtype != torch.float32:
            raise ValueError('Fused recurrence requires CPU float32 inputs')
        arrays = [v.detach().contiguous().numpy() for v in (pre, ww, wr, wc)]
        hs, w, r, candidate = _forward(*arrays)
        output = torch.from_numpy(hs)
        ctx.save_for_backward(output, torch.from_numpy(w), torch.from_numpy(r),
                              torch.from_numpy(candidate), ww, wr, wc)
        return output

    @staticmethod
    def backward(ctx, grad):
        hs, w, r, candidate, ww, wr, wc = ctx.saved_tensors
        arrays = [v.detach().contiguous().numpy() for v in
                  (grad, hs, w, r, candidate, ww, wr, wc)]
        gp, gww, gwr, gwc = _backward(*arrays)
        return (torch.from_numpy(gp), torch.from_numpy(gww.sum(axis=0)),
                torch.from_numpy(gwr.sum(axis=0)), torch.from_numpy(gwc.sum(axis=0)))


def _fused_block_forward(self, x):
    if x.device.type != 'cpu' or x.dtype != torch.float32:
        raise ValueError('The prototype supports CPU float32 only')
    u = self.pre(x)
    p = self.history(u)
    d = self.width
    packed = torch.cat([

        torch.cat((weight[:, :d], weight[:, 2 * d:]), dim=1)
        for weight in (self.Ww, self.Wr, self.Wc)
```

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

Fused CPU execution for ILRM blocks

Packed projections, Numba forward/backward and opt-in wrapper

ilrn_fused_cpu.py Lines 121-135 of 135

```
    ], dim=0)
    bias = torch.cat((self.bw, self.br, self.bc), dim=0)
    pre = F.linear(torch.cat((u, p), dim=-1), packed, bias)
    state = _FusedRecurrence.apply(pre,
                                   self.Ww[:, d:2 * d],
                                   self.Wr[:, d:2 * d],
                                   self.Wc[:, d:2 * d])
    return self.post(x + state)


def enable_fused_cpu(model):
    """Patch only the forward implementations. All state_dict keys remain the same."""
    for block in model.blocks:
        block.forward = MethodType(_fused_block_forward, block)
    return model
```

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

ILRN numerical and gradient tests

Matching-weight logits, gradients and optimiser-update checks

verify_ilrn.py Lines 1-40 of 54

```
"""Minimal numerical and checkpoint-compatibility test; run after installation."""
import copy
import numpy as np
import numba
import torch
from torch.nn import functional as F
from ilrn_reference import StackedILRN
from ilrn_fused_cpu import enable_fused_cpu


def verify(context=32, batch=2):
    torch.manual_seed(20261011)
    torch.set_num_threads(2)
    numba.set_num_threads(2)
    original = StackedILRN()
    accelerated = enable_fused_cpu(copy.deepcopy(original))
    x = torch.randint(0, 1024, (batch, context))
    y = torch.randint(0, 1024, (batch, context))
    result = {}
    # Check identical parameter identifiers and shapes before gradient comparisons.
    assert original.state_dict().keys() == accelerated.state_dict().keys()
    for label, model in [('normal', original), ('fused', accelerated)]:
        model.zero_grad(set_to_none=True)
        logits = model(x)
        loss = F.cross_entropy(logits.reshape(-1, 1024), y.reshape(-1))
        loss.backward()
        gradients = {k: v.grad.detach().clone()
                     for k, v in model.named_parameters()}
        result[label] = (logits.detach(), gradients, float(loss.detach()))
    a, ga, la = result['normal']
    b, gb, lb = result['fused']
    max_logit = (a-b).abs().max().item()
    max_grad = max((ga[k]-gb[k]).abs().max().item() for k in ga)
    agree = (a.argmax(-1) == b.argmax(-1)).float().mean().item()
    assert torch.allclose(a, b, atol=3e-4, rtol=3e-4)
    assert max_grad < 3e-4
    assert abs(la-lb) < 3e-4
    # Check one optimising step from equal initial weights.
    oa = torch.optim.AdamW(original.parameters(), lr=0.001, weight_decay=0.01)
    ob = torch.optim.AdamW(accelerated.parameters(), lr=0.001, weight_decay=0.01)
```

CODE APPENDIX / REPRODUCIBLE IMPLEMENTATION

ILRN numerical and gradient tests

Matching-weight logits, gradients and optimiser-update checks

verify_ilrn.py Lines 41-54 of 54

```
    oa.step(); ob.step()
    weight_diff = max((dict(original.named_parameters())[k] -
                       dict(accelerated.named_parameters())[k]).abs().max().item()
                      for k in ga)
    assert weight_diff < 3e-4, 'Large one-step optimiser drift'
    print(f'context={context} logits max diff={max_logit:.3g}; '
          f'grad max diff={max_grad:.3g}; top-1 agreement={agree:.2%}; '
          f'one-step parameter max diff={weight_diff:.3g}')
    return max_logit, max_grad, weight_diff


if __name__ == '__main__':
    verify(32, 2)
    verify(96, 2)
```

ILRM LICENSING / ILRN COMPOSITION

What is licensed, and what is built from it

12 October 2026 | Independent researcher | rickeyernest1990@gmail.com

ILRN-4-96 = the network built from four ILRM memory blocks

The four ILRM blocks have separate learned weights and recurrent states. Embeddings, pre/post LayerNorm, residual connections and an output head complete the ILRN network.

ILRM block 1 › ILRM block 2 › ILRM block 3 › ILRM block 4

ORIGINAL ILRM MATERIALS: SPLIT NONCOMMERCIAL LICENSING

1. ILRM documentation | CC BY-NC 4.0

The original ILRM publication places its copyrightable explanatory text, figures, tables and presentation under Creative Commons Attribution-NonCommercial 4.0 International.

creativecommons.org/licenses/by-nc/4.0/

2. ILRM implementation code | PolyForm Noncommercial 1.0.0

The original ILRM licence covers source code implementing ILRM, including ILRM code listings used as blocks in the four-layer ILRN implementation. Embedding ILRM source code in ILRN does not remove its applicable conditions.

polyformproject.org/licenses/noncommercial/1.0.0/

Rights boundary

These licence notices originate from the ILRM publication dated 31 August 2026. ILRN names the four-layer composition; it is not a replacement licence for ILRM, nor does this page declare one blanket licence for every distinct ILRN-only contribution. Neither ILRM licence grants unrestricted commercial use.

Source and scientific scope

The original ILRM equations were compared with the newer four-layer implementation. This licensing applies to relevant protected expression and software, not mathematical facts, formulae or abstract ideas. Benchmark results remain sourced observations, not newly rerun experiments.